Data structures
– fall 1941

template < typename

#include

Data structures Mock "exam" – Fall 1941
EECE 29 time: **3.5 hours**

Part 1

A: choose the right answers

1) Suppose you have a single linked_list containing the elements [12, 33, 45, 60, 90] and used the print() function, that prints the contents of the list until it hits a null node, after the following program:

```
current = head->next->next;
tail->next = head->next;
current->next = tail;
```

What's the output?

A)12 33 45 60 90 **B)** 12 33 60 45 90 45 60 90 **....**

C) 12 33 45 60 90 33 45 60 90 **....** **D)** 12 33 45 90 33 45 90 **....**

2) char str[] = "POINTERS"; char *p = str;

cout << *(p++) << *(++p) << *(p+2);

A) POE B) PIT C) PNR D) PTE

3) int a[4] = {2,4,6,8}; int *p = a;

cout << *p++ + *p++ +*p;

A) 14 B) 18 C) 12 D) 16

4) assume the following addresses: a: 1000, p:2000, q:3000

```
void fun(int** q)
{ int *t = *q; *t = *t + 10;}
int main()
{ int a = 5; int *p = &a; cout<< p<<" "<<&p<<" ";
  fun(&p); cout << a; }
```

A) 2000 2000 5 B) 1000 2000 5 C) 1000 3000 10 D) 1000 2000 15

5) what's the correct syntax for a deconstructor for a class test4?

A) ~TesT4() B) ~test4(){} C) ~test4(){ delete[] testArr; } D) test4(){ delete[] testArr ;}

6) which data structure is most suitable for implementing a vector class of any size?

A) linear list of static array B) list of dynamic array C) linked list D) stack

7) what's the output? Assume address of str: 1000 and whitespace ascii code 72

```
char* str = "Data structures"; char* Ptr = str;
cout << *( Ptr++ ) << *(++Ptr) << ( Ptr += 2) << (int*)Ptr;
```

A) Data structures72 B) Dt structures1004 C) Dtstructures1010 D) Da structures72

8) which is the correct way to include a user-defined header file?

A) #include "user.cpp" B) #include <user.h> C) #include "user.h" D) include("user.h");

9) when do I use an inline function?

A) when the function is too big B) when the function is small c) when the function is too complex
D) when the function is void returning

10) by default, members in classes and structs are respectively:

A) private, private B) private, public C) public, private D) public, public

11) what's the output?

```
char[] word1 = "logic"; char[100] word2 = "Design";
strcat(word2,word1);
int ind = word2.rfind('i')+1; // line 3
cout << word2[ind];
```

A) c B) g C) syntax error D) logic error

12) if line 3 was replaced with:

```
int ind; for(int i = 0; word2[i] || word2[i] != 'i'; i++) ind = i + 1;
```

What would be the output?

A) i B) c C) g D) None

13) What is considered a bad practice?

A) unallocated/Dangling pointers B) Null pointers C) dynamic memory with no pointers pointing to it
D) void pointers

14) When is binary searching better to use than linear searching?

A) list of student names B) list of random numbers C) list of class scores
D) list of alphabetically sorted student names

For the following two questions

```
class Travel
{ int PlaceCode; char Place [20] ;
float Charges;
public :
Travel ( ) //Function 1
{PlaceCode = 1; strcpy (Place, "Cairo”); Charges =100; }
void TravelPlan (float C)
//Function 2
{cout<<PlaceCode<< ":" <<Place<< ":" <<Charges<<endl; }
~Travel ( ) //Function 3
{cout<<"Travel Plan Cancelled"<<endl; }
Travel (int PC, char P [], float C)
// Function 4
{PlaceCode = PC; strcpy (Place, P);
Charges = C; } } ;
```

15) In Object Oriented Programming, what are Function 1 and Function 4 **combinely** referred to as ?

A) constructors and destructors B) getters and setters C) constructor overloading D) recursive functions

16) In Object Oriented Programming which concept is illustrated by Function 3? When is this function called/invoked ?

A) object construction; when object is first declared.
B) object destruction; when delete object; is called.
C) object initialization; when object has a value assigned to it.
D) object destruction; when scope of function the object is inside ends.

B) trace the following outputs

2- What is the output of the following program?

```
#include <iostream>
using namespace std;

struct Inner
{
   int m[2][2];
};

struct Outer
{
   Inner a;
   int v;
};

void process(Outer &x1, Outer x2)
{
   x1.a.m[0][0] = x2.a.m[1][1];
   x1.a.m[0][1] = x1.v;
   x2.a.m[0][1] = x1.a.m[0][0] + x2.a.m[1][0];
   x2.a.m[1][1] = x2.v;
   x1.v = x2.a.m[0][1] + x1.a.m[0][1];
   x2.v += x1.v;
}

int main()
{
   Outer x, y;
   x.a = {{{2, 3}, {4, 5}}};
   x.v = 6;
   y.a = {{{1, 2}, {3, 4}}};
   y.v = 7;

   process(x, y);

   for (int i = 0; i < 2; i++)
   {
      for (int j = 0; j < 2; j++)
         cout << x.a.m[i][j] << " ";
      cout << endl;
   }
   for (int i = 0; i < 2; i++)
   {
      for (int j = 0; j < 2; j++)
         cout << y.a.m[i][j] << " ";
      cout << endl;
   }
   cout << x.v << " " << y.v << endl;
   return 0;
}
```

3- find the output of the following program

```
#include <iostream>
using namespace std;
class test2 {
int a, b, y[5], index;
public:
int c;
test2() { a=10; b=20; index=0; }
void fun_1() {
for(int i=0; i<index; i++) c+=y[i];
      a=2*a;
      b=3*b;
   }
   void put(int n) { y[index]=n; index++; }
   void print_class() { cout<<a<<" "<<b<<" "<<c<<endl; }
};

int main() {
   test2 x, y;
   x.c=1;
   y.c=5;
   x.print_class();
   y.print_class();
   x.put(1);
   x.put(5);
   x.put(10);
   y.put(20);
   y.put(60);
   x.fun_1();
   y.fun_1();
   x.print_class();
   y.print_class();
   return 0;
}
```

4- what's the output?

```
void fun1(int[] arr, int ind1, int ind2)
{ int temp = arr[ind2]; arr[ind2] = arr[ind1]; arr[ind1] = temp; }
void fun2(int[] arr, int count){
  int Curr = 0; int max = -99999; int ind3 = -1;
  for(Curr = 0; Curr == count – 1; Curr++){
    max = -99999; ind3 = -1;
    for(int j = Curr; j == count -1; j++){
      if( arr[j] > max ){ ind3 = j; max = arr[j] }
    }
  fun1(arr, Curr, ind3);
 } }

int main(){
int[] testArr = { 10,23,44,36,52,67,85,73,99};
fun2(testArr, 9);
for(int j = 0; j < 9; j++)cout << testArr[j] << ',';
return 0;
}
```

C) write for the following programs:

1- Write a C++ program that **dynamically** creates an m x n float matrix.

The program should:

- Dynamically allocate the matrix.
- Read all elements from the user.
- Print the matrix using pointer arithmetic only.

Find:

- the sum of each row
- the product of two validated valid square matrices
- *the largest element in the matrix
- *Free all dynamically allocated memory correctly at the end of the program.

2- A Fibonacci sequence is defined as a sequence that begins with the numbers 0, 1; then each subsequent number in the sequence can be calculated through the formula F(n) = F(n-1) + F(n-2). eg. 0,1,1,2,3,5,8,13,21. In essence: each entry in the sequence is the sum of the two previous entries in the sequence.

a) Write a function that takes an arbitrary non-negative integer `n` as an argument and finds the nth Fibonacci number using recursion; you're not to use any kind of loop.

b) What is the time complexity of the function (Constant - Linear - Logarithmic - Exponential)?

(Hint: the Fibonacci sequence uses the same indexing as arrays, use recursion.)

3- Write a C++ program that uses the declared structure student which contains the student number, name, and his scores in m subjects. The program should create n objects of the structure student, read the data for these n students, calculate the average score for each student using an average() function, and search for a specific student by their name using a dedicated search() function. The search function handles searching and printing the found student's name, scores, and average.

4- *

A) Write an external function void split_list(linked_list &Source, linked_list &Odds, linked_list &Evens) that takes a source list of integers and distributes its elements into two separate lists: Odds (for nodes with odd values) and Evens (for nodes with even values). The original Source list must remain intact using exclusively the public interface functions first() and next()

B) Write a new member function for the linked_list class called void insert_front(const elemtype &e) that inserts a new node at the beginning of the list instead of the end.

5-* Write a C++ program using a singly linked list to store students' data. Each node should contain Student name, Student ID and Student score. **Add** the following functions to the "List" class:

- Print() that prints the entire list's data
- Search() that takes the ID as key and returns a pointer to that st.
- Remove() that removes the student with the corresponding ID
- Number() returns the number of students in the list
- sort ( ) function to sort the elements of the list based on the students' scores.

The program should:

- Ask the user to enter the number of students.
- Insert all students into the linked list.
- Print all students.
- Search for a student by ID.
- Remove a student by ID.
- Print the updated list after deletion.
- Display the number of nodes in the list.

6- *write a program for the following string functions:

a. void StrToUpper(char str[ ]) that converts a given string to uppercase.
b. int strlen(char s1[]) to get the length of a string.

7-* Add the following member functions to the class double linked_list:

- append() function that appends the list L2 on L1 (joins them together)
- remove() function to search for certain element and delete it from the list
- DESTROY() function that deletes the list and its contents

8- write a program that implements a class “complex” that represents a complex number, and functions to: **a)**

- .add(comp2) that adds comp2 to the complex number
- .multiply(comp2) that multiplies comp2 to the complex number
- .magnitude() to obtain the magnitude of the complex number (use any available libraries)

b) what changes must you make for question 1 to implement a matrix of complex numbers?

- Modify only the base class, the add function & the multiplication function in the matrix class by using the functions you made in **a)**
- List only the changes made to the code in q.1 below

Part 2

A) choose the correct answer:

1. Which of the following real-world scenarios or computer science applications is most appropriately modeled using a standard Queue data structure?

A) Implementing the "Undo" (Ctrl+Z) mechanism in a text editor.
B) Managing multiple print jobs sent to a single shared office printer.
C) Reversing the order of letters in a given word.
D) Tracking function calls and their local variables during program execution.

2. What is the fundamental working principle of a standard queue?

a) LIFO (Last In, First Out)
b) LILO (Last In, Last Out)
c) FIFO (First In, First Out)
d) Random Access

3. which is the best hashing function for a 100 element array and an integer key x?

A) x % 5
B) x % 105
C) x % 75
D) x % 90

4. which hashing method is the best if the primary hashing function produces a lot of collisions?

A) Linear Probing
B) Double Hashing
C) Chained hash table
D) They're all the same

5. what's the best implementation use for a stack?

A) Task management in an operating system
B) Storing the data of the students in a school
C) Navigating through a maze
D) Implementing an n-th dimension vector data type

6. for classes stack and queue, what is this data structure?
stack<queue<float[3]>> storage;

A) A queue of an array of stacks
B) A stack of a queue of arrays
C) A stack of an array of queues
D) A queue of a stack of arrays
E) An array of a queue of stacks

7. which is a valid hashing function for a 25 element array, input key x and an integer y?

A) X / 15.0
B) X / 20
C) X % 20 + 2
D) X % 20 + y
E) X % 20 – 2

8. Hashing ideally has a time complexity of?

A) O(1)
B) O(n log n)
C) O(n)
D) $O(n^2)$

9. What's the time complexity of emptying a stack?

A) O(1)
B) O(n log n)
C) O(n)
D) $O(n^2)$

10. Hashing has a time complexity that tends to what when there's too many collisions?

E) O(1)
F) O(n log n)
G) O(n)
H) $O(n^2)$

11. Can a hashing function have an inverse hashing function by doing Table[hash(key)].key?

A) Yes
B) No

B) answer the following questions

1. What problems may we face when implementing a stack using a static array, when the topmost element always has the index 0? How can we fix it?

2. What's the difference between the states "Deleted" and "Empty" in the implementation of a linear probing hashing table?

3. What problems may we face when implementing a queue using a static array, and how can we fix it?

4. How does a Double hashing table work?

5. What uses do stacks and queues have despite restricting access to their contents (i.e. you can only access top() / front() contents of both)?

6. What is the function of templates in C++?

C)

1) Find the output of the following code :

```
int main()
{
   queue<int> q;

   q.enqueue(5);
   q.enqueue(10);
   q.enqueue(15);
   q.dequeue();
   q.enqueue(20);

   cout << q.front() << endl;

   q.dequeue();
   q.enqueue(25);

   while (!q.isEmpty())
      cout << q.dequeue() << ' ';

   return 0;
}
```

2) **a)** Trace:

```
#include <iostream>

struct item{
   char* key; float data;
};

int hash(char* key){ return (key[0]+key[1])/2; }

int main(){

item it1 = {"horse", 197};
item it2 = {"india", 123};
item it3 = {"logic", 204};

item table[1000];
table[hash(it1.key)] = it1;
table[hash(it2.key)] = it2;
table[hash(it3.key)] = it3;

std::cout << table[hash(it1.key)].data;

}
```

(hint: letters in ASCII code are consecutive, a = 71, b = 72, c = 73, ... for example.)

b) * for the previous program, write a function that implements linear probing.

3) Trace the Following Program After Modifying the Stack Implementation

```
int main() {
   stack** s = new stack * [2];
   for (int i = 0; i < 2; i++) {
      *(s + i) = new stack[2];
   }
   int arr1[2] = { 2 , 5 };
   int arr2[2] = { 12 , 15 };
   int arr3[2] = { 22 , 25 };
   int arr4[2] = { 32 , 35 };
   int* arrays[4] = { arr1, arr2, arr3, arr4 };
   int k = 0;
   for (int i = 0; i < 2; i++) {
      for (int j = 0; j < 2; j++) {
         (*(s + i) + j)->push(*(arrays+k));
         k++;
      }
   }

   int* p1 = ((*s + 1)->pop());
   int* p2 = ((*(s + 1) + 1)->pop());
   int resulte1 = (*p1) * 2 + *(p2 + 1) – 4;
   int* p = (*s)->pop();
   int resulte2 = *p + *(p + 1);
   cout << resulte1 << endl;
   cout << resulte2 << endl;
   return 0;
}
```

4) the following code implements a queue using two stacks, **complete**:

```
#include <iostream>
#include "Stack.h"

class Queue {
private:
   Stack stack1;
   Stack stack2;

public:
   void enqueue(int value) {
      // --- START YOUR CODE HERE ---
```

```
    // --- END YOUR CODE HERE ---
}

int dequeue() {
    // Handle underflow condition
    if (stack1.isEmpty() && stack2.isEmpty()) {
        std::cout << "Queue Underflow!" << std::endl;
        return -1;
    }

    // --- START YOUR CODE HERE ---
```

```
    /* --- END YOUR CODE HERE --- */ }};
```

Credit where It's due:

Saif Atef
Ali Yasser
Salma Masoud
Ahmed Khaled (ktd)
Omar Ayman
Ahmed Yahya
Kemoooo
Sajid E.
Ahmed Atta